LP-III Blockchain Technology

Practical Study Guide: Experiments 1, 3 and 4

Theory, Wallet and Faucet Procedure, Solidity Code, Testing, Gas, Troubleshooting and Viva Questions

Contents

1 How to Use This Guide **3**
- 1.1 Websites and Tools . . . 3
- 1.2 Using a Google-accessible Sepolia Faucet . . . 3

2 Background Theory Needed for Both Experiments **4**
- 2.1 Blockchain . . . 4
- 2.2 Smart Contracts and the EVM . . . 4
- 2.3 Solidity Data Locations . . . 4
- 2.4 Ether, Wei and Gas . . . 4
- 2.5 Local VM Versus Public Testnet . . . 4

3 Experiment 1: Installation of MetaMask and Study of Ether Spent per Transaction **5**
- 3.1 Aim . . . 5
- 3.2 Theory . . . 5
- 3.3 Websites and tools . . . 5
- 3.4 Procedure . . . 5
- 3.5 Illustrative workflow diagram . . . 6
- 3.6 Observation table . . . 6
- 3.7 Precautions and common issues . . . 7
- 3.8 Viva questions and answers . . . 7

4 Experiment 3: Bank Smart Contract **8**
- 4.1 Aim . . . 8
- 4.2 Problem Statement . . . 8
- 4.3 Theory . . . 8
- 4.4 Solidity Program: `Bank.sol` . . . 8
- 4.5 Code Explanation . . . 9
- 4.6 Step-by-Step Procedure in Remix VM . . . 9
- 4.7 Expected Observations . . . 10
- 4.8 Common Errors and Fixes . . . 10
- 4.9 Experiment 3: Viva Questions with Answers . . . 10

5 Experiment 4: Student Data Using Struct, Array and Fallback **12**
- 5.1 Aim . . . 12
- 5.2 Theory . . . 12
- 5.3 Solidity Program: `Student.sol` . . . 12
- 5.4 Code Explanation . . . 13
- 5.5 Step-by-Step Procedure in Remix VM . . . 13
- 5.6 Gas and Transaction Observation . . . 14
- 5.7 Expected Observations . . . 14
- 5.8 Experiment 4: Viva Questions with Answers . . . 14

6 Quick Revision Tables **15**
6.1 Important Solidity Terms 15
6.2 Useful Conversions 16

7 Final Practical Checklist **16**

1 How to Use This Guide

This guide covers the two smart-contract experiments discussed in the practical notes:

1. A Bank smart contract with deposit, withdrawal and balance checking.
2. A Student-data smart contract using a structure, a dynamic array and fallback handling.

The code is a modern teaching implementation using Solidity `^0.8.20`. If the official laboratory manual prescribes different code or function names, follow the manual for formal submission and use this guide to understand the concepts.

1.1 Websites and Tools

- Remix IDE: https://remix.ethereum.org/
- MetaMask wallet: https://metamask.io/
- Google Cloud Web3 faucet for Ethereum Sepolia test ETH: https://cloud.google.com/application/web3/faucet/ethereum/sepolia
- Sepolia Etherscan: https://sepolia.etherscan.io/
- Ethereum developer documentation: https://ethereum.org/en/developers/docs/

Recommended workflow: first compile and test in Remix VM. Remix VM is a simulated local blockchain and does not require Ether. For a public testnet demonstration, use MetaMask with Sepolia and obtain test ETH from the Google Cloud Web3 faucet linked above, if you meet its current eligibility requirements. Faucet rules can change and may require an eligible account or minimum mainnet balance; this is a faucet policy, not a universal Ethereum requirement. Never share a Secret Recovery Phrase or private key, and do not buy real ETH just to complete a lab exercise.

1.2 Using a Google-accessible Sepolia Faucet

1. Open the Google Cloud Web3 faucet link above and read the current eligibility and usage instructions.
2. Confirm that the destination address is your MetaMask **Sepolia** account address. A public address can receive test ETH; never enter or disclose your Secret Recovery Phrase or private key.
3. If the faucet requires sign-in, verification or a minimum mainnet balance, follow its displayed policy. If you are not eligible, use another faucet approved by your instructor or use Remix VM for local practice. Do not purchase real ETH solely to qualify for a faucet.
4. Request Sepolia test ETH and wait for the faucet to confirm completion. The amount and request limits depend on the faucet's current policy.
5. In MetaMask, select the Sepolia network and check the test ETH balance. Test ETH has no intended real-world monetary value.
6. When deploying through Remix, select *Injected Provider* and verify that MetaMask and Remix both show Sepolia before confirming a transaction.
7. After a transaction is mined, copy its transaction hash and search it on https://sepolia.etherscan.io/.

Important: the faucet is used to obtain test ETH for Sepolia gas fees. It is not required when using Remix VM. Do not confuse Sepolia test ETH with ETH on Ethereum Mainnet.

2 Background Theory Needed for Both Experiments

2.1 Blockchain

A blockchain is a ledger replicated across participating computers or nodes. Data is grouped into blocks, and cryptographic links help make changes detectable. The exact governance, access rules and consensus method depend on the blockchain. Not every blockchain is public or permissionless.

2.2 Smart Contracts and the EVM

A smart contract is code deployed to a blockchain. On Ethereum-compatible networks, contract bytecode is executed by the Ethereum Virtual Machine (EVM). Contract state can persist between transactions. A transaction that changes state is ordered and processed by the network and normally requires gas.

2.3 Solidity Data Locations

- `storage`: persistent state held by the contract.
- `memory`: temporary data used during a function call.
- `calldata`: read-only input data for external function calls; commonly used for external function parameters when appropriate.

2.4 Ether, Wei and Gas

Ether (ETH) is Ethereum's native asset. One ETH equals 10^{18} Wei; one ETH equals 10^{9} Gwei. Gas measures computational work. Gas used is the amount actually consumed; the gas limit is the maximum permitted for the transaction. A simplified fee formula is

$$\text{Transaction fee} = \text{gas used} \times \text{effective gas price}.$$

When the gas price is expressed in Gwei, divide the product by 10^{9} to convert the fee to ETH. The amount transferred does not by itself determine the fee. Code complexity, storage changes, execution path and network conditions can affect it. Failed transactions that are included on-chain can still consume gas.

2.5 Local VM Versus Public Testnet

- **Remix VM:** a local simulation for development and practice. Its accounts and results are not public Ethereum transactions.
- **Sepolia:** a public Ethereum test network. Transactions can be inspected on Sepolia Etherscan and use test ETH for gas.

3 Experiment 1: Installation of MetaMask and Study of Ether Spent per Transaction

3.1 Aim

To install and configure MetaMask, obtain Sepolia test ETH using a faucet when eligible, perform a test transaction, and study the Ether amount and transaction fee.

3.2 Theory

MetaMask is a crypto wallet interface and browser extension/mobile wallet. It manages wallet accounts, stores or protects access to the private keys locally, signs transactions after user approval, and connects to decentralised applications (dApps). It does not itself store coins on a server or operate the Ethereum blockchain.

Ether (ETH) is Ethereum's native asset. Gas is the unit used to measure the computational work required by an Ethereum transaction. A transaction's fee depends on gas used and the effective gas price, not simply on the amount of ETH transferred.

- 1 ETH = 10^{18} Wei.
- 1 Gwei = 10^9 Wei, so 1 ETH = 10^9 Gwei.
- **Gas used**: actual computational units consumed.
- **Gas limit**: maximum gas the sender allows the transaction to consume.
- **Gas price**: price paid per unit of gas, commonly displayed in Gwei.
- **Transaction fee**: gas used multiplied by the effective gas price.

$$\text{Fee (ETH)} = \frac{\text{Gas Used} \times \text{Gas Price (Gwei)}}{10^9}$$

For example, if a simple transfer uses 21,000 gas and the effective gas price is 20 Gwei, the fee is $21{,}000 \times 20/10^9 = 0.00042$ ETH. This is an illustrative calculation, not a claim about the fee for every transaction.

Mainnet versus testnet: Ethereum Mainnet uses real ETH with real monetary value. Sepolia is a test network; Sepolia test ETH is intended for development and practical exercises and has no intended real-world monetary value. A faucet distributes test ETH subject to its own current eligibility rules. A faucet asking for a minimum mainnet balance is imposing its own policy; it is not a universal Ethereum requirement. Do not buy real ETH merely to qualify for a lab faucet.

3.3 Websites and tools

- MetaMask: https://metamask.io/
- Google Cloud Web3 faucet for Sepolia: https://cloud.google.com/application/web3/faucet/ethereum/sepolia
- Sepolia Etherscan: https://sepolia.etherscan.io/
- Remix IDE (useful for later smart-contract experiments): https://remix.ethereum.org/

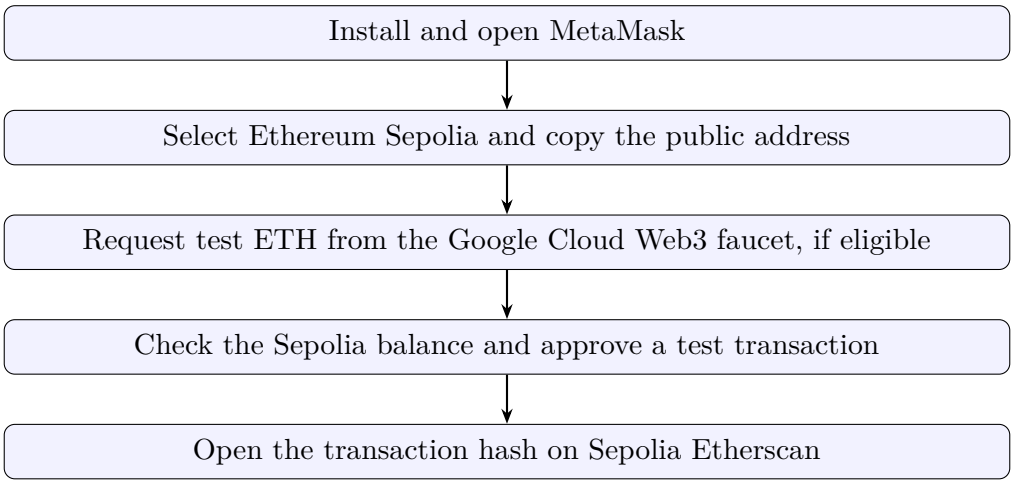
3.4 Procedure

1. Install MetaMask only from its official website or official app-store listing. Create a new wallet or import an existing wallet only if instructed.
2. Set a strong wallet password. Write down the Secret Recovery Phrase offline and store it securely. Never share it with a person, website, support agent, or chatbot.

3. Open MetaMask and select the Ethereum Sepolia test network. If Sepolia is not shown, use MetaMask's network settings to enable or add the test network using the current official guidance.
4. Copy the public account address from MetaMask. A public address may be shared to receive test ETH; the Secret Recovery Phrase and private key must not be shared.
5. Open the Google Cloud Web3 Sepolia faucet link above. Read the current rules and connect/sign in only if required by the faucet.
6. Request Sepolia test ETH to the MetaMask public address, provided you satisfy the faucet's current requirements. The amount, limits and eligibility checks can change. If blocked, record the displayed reason and ask the instructor for an approved alternative; do not purchase mainnet ETH for this practical.
7. Return to MetaMask, confirm that the selected network is Sepolia, and check the test ETH balance.
8. For the transaction demonstration, send a small amount of Sepolia test ETH to a recipient address supplied by the instructor or another test account. Confirm the network, recipient, amount and estimated fee before approving.
9. After submission, copy the transaction hash and open it on Sepolia Etherscan. Record status, value, gas used, effective gas price and transaction fee where available.
10. Compare the fee with a second transaction if required. Explain differences using gas used, transaction complexity and gas price/network conditions rather than transferred amount alone.

3.5 Illustrative workflow diagram

This is a schematic guide, not a screenshot of the live interface. Website layouts and faucet rules may change.



3.6 Observation table

Fill this table from your own transaction page; do not copy illustrative values as actual observations.

Observation	**Value recorded from MetaMask / Etherscan**
Network	Sepolia testnet
Transaction hash	
Transaction status	
Value transferred (ETH)	
Gas used	
Effective gas price (Gwei)	
Transaction fee (ETH)	

3.7 Precautions and common issues

- Verify that the network is Sepolia before requesting test ETH or sending a test transaction.
- Never disclose the Secret Recovery Phrase or private key. Genuine support should not ask for them.
- Do not assume all faucets have the same rules. A minimum mainnet-balance message is specific to that faucet.
- A pending transaction may take time to be included. A transaction that fails after being included may still consume gas.
- Check the transaction hash on the correct explorer: Sepolia transactions belong on Sepolia Etherscan, not the Mainnet explorer.
- Remix VM is a local simulated blockchain and does not require faucet funds; public Sepolia deployment does require Sepolia test ETH for gas.

3.8 Viva questions and answers

1. **What is MetaMask?** A wallet interface used to manage Ethereum accounts, sign transactions and connect to dApps.
2. **Does MetaMask physically store ETH?** No. Blockchain state records balances; MetaMask manages access to accounts and transaction signing.
3. **What is a wallet address?** A public identifier used to receive assets and identify an account on the network.
4. **What is a private key?** Secret cryptographic material that authorises account actions; it must remain confidential.
5. **What is a Secret Recovery Phrase?** A set of words used to recover wallet access. Anyone who obtains it may gain control of the wallet.
6. **What is Ether?** Ethereum's native asset, commonly abbreviated ETH.
7. **What is Wei?** The smallest denomination of Ether; one ETH equals 10^{18} Wei.
8. **What is Gwei?** A denomination equal to 10^{9} Wei, commonly used to express gas prices.
9. **What is gas?** A measure of computational work consumed by an Ethereum transaction or smart-contract execution.
10. **What is gas used?** The actual amount of gas consumed by an executed transaction.
11. **What is a gas limit?** The maximum gas the sender permits the transaction to consume.
12. **How is the transaction fee calculated?** Gas used multiplied by the effective gas price.
13. **Does sending more ETH always mean paying a higher fee?** No. The fee mainly depends on gas consumed and gas price, not the transferred value.
14. **What is Ethereum Mainnet?** The primary public Ethereum network where ETH and transactions have real economic value.
15. **What is Sepolia?** An Ethereum testnet used to test applications and contracts without using Mainnet ETH.
16. **What is a faucet?** A service that distributes testnet tokens subject to its rules and limits.
17. **Is Sepolia test ETH real money?** It is intended for testing and has no intended real-world monetary value.

18. **Why might a faucet reject a request?** It may impose rate limits, identity checks, account restrictions or a minimum-balance eligibility rule.
19. **What is Etherscan used for?** It is a block explorer for checking transactions, blocks, addresses, status, gas and fees.
20. **What is a transaction hash?** A unique identifier used to look up a transaction on the relevant blockchain explorer.
21. **Why can a failed transaction still cost gas?** If included on-chain, computation was performed before failure and the consumed gas is still charged.
22. **What is the difference between Remix VM and Sepolia?** Remix VM is a local simulated environment; Sepolia is a public testnet with network transactions and test ETH gas fees.
23. **Why should the network be checked before sending?** The same wallet may access different networks, and transactions or balances are network-specific.
24. **Can the Secret Recovery Phrase be entered into a faucet?** No. A faucet should only need a public address and any legitimate eligibility steps it specifies.

4 Experiment 3: Bank Smart Contract

4.1 Aim

To write, compile, deploy and test a Solidity smart contract that supports depositing Ether, withdrawing Ether and checking the caller's recorded balance.

4.2 Problem Statement

Create a contract that records a separate internal balance for each Ethereum address. A user should be able to deposit Ether, withdraw an amount not exceeding their recorded balance, and view their recorded balance.

4.3 Theory

1. **Mapping:** `mapping(address => uint256)` associates an address with an unsigned integer. The public mapping generates a getter that takes an address.
2. **Payable function:** a function marked `payable` can receive Ether with the call.
3. **`msg.sender`:** the address that called the function.
4. **`msg.value`:** the amount of Ether attached to the current call, represented in Wei.
5. **`require`:** checks a condition and reverts execution if the condition is false.
6. **`view`:** declares that a function reads state but does not modify it.
7. **Withdrawal pattern:** validate inputs and balance, update the internal balance, then make the external transfer. Updating state before the external call follows the checks-effects-interactions pattern and helps reduce reentrancy risk.

4.4 Solidity Program: `Bank.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Bank {
    mapping(address => uint256) public balanceOf;
```

```
    function deposit() public payable {
        require(msg.value > 0, "Amount must be greater than zero");
        balanceOf[msg.sender] += msg.value;
    }

    function withdraw(uint256 amount) public {
        require(amount > 0, "Amount must be greater than zero");
        require(amount <= balanceOf[msg.sender], "Insufficient balance");

        // Update the recorded balance before the external transfer.
        balanceOf[msg.sender] -= amount;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Transfer failed");
    }

    function showBalance() public view returns (uint256) {
        return balanceOf[msg.sender];
    }
}
```

4.5 Code Explanation

- `pragma solidity ^0.8.20;` declares the compiler version range.
- `mapping(address => uint256) public balanceOf;` stores a recorded balance per address.
- `deposit()` is payable because the caller sends Ether with the transaction.
- `require(msg.value > 0, ...)` rejects a zero-value deposit.
- `balanceOf[msg.sender] += msg.value;` adds the received amount to the caller's recorded balance.
- `withdraw(uint256 amount)` receives the requested withdrawal amount as a function argument in Wei.
- The two `require` statements reject zero withdrawals and withdrawals exceeding the recorded balance.
- `payable(msg.sender).call{value: amount}("")` attempts to send Ether back to the caller. The return value indicates whether the transfer succeeded.
- `showBalance()` returns the caller's recorded balance. The value returned is in Wei, not a formatted ETH value.

4.6 Step-by-Step Procedure in Remix VM

1. Open https://remix.ethereum.org/.
2. In File Explorers, create a file named `Bank.sol`.
3. Paste the program above and save it.
4. Open *Solidity Compiler*; select a compiler compatible with `^0.8.20`; compile `Bank.sol`.
5. Open *Deploy & Run Transactions*. Select *Remix VM* as the environment and select the `Bank` contract.
6. Keep the deployment *Value* at `0 Wei`, then click *Deploy*.
7. Under *Deployed Contracts*, set the transaction *Value* to `1 Ether` and call `deposit()`.

8. Reset the top-level transaction *Value* to `0 Wei`. Call `showBalance()` to read the caller's recorded balance.
9. To withdraw half an Ether, enter `500000000000000000` in the `withdraw(amount)` argument box. Keep the top-level transaction *Value* at `0 Wei`, then execute `withdraw()`.
10. Call `showBalance()` again and compare the result.
11. Try withdrawing more than the recorded balance to confirm that the transaction reverts.

4.7 Expected Observations

- If the caller starts with no recorded balance and deposits 1 ETH, `showBalance()` returns `1000000000000000000` Wei.
- After withdrawing 0.5 ETH, the recorded balance should be `500000000000000000` Wei.
- A zero-value deposit reverts with the specified error.
- A withdrawal above the recorded balance reverts with `Insufficient balance`.

4.8 Common Errors and Fixes

Error or symptom	Likely cause and fix
Deployment fails or appears too expensive	Set deployment Value to 0 Wei. Do not attach Ether while deploying this sample.
Invalid BigNumberish or empty amount	Enter the withdrawal amount in the `withdraw()` argument field. Do not enter it in the top-level Value field.
Invalid address / empty input	The generated `balanceOf` getter needs an address argument. `showBalance()` does not.
Zero deposit reverts	Expected behavior: the code requires `msg.value > 0`.
Insufficient balance	The requested withdrawal is greater than the caller's recorded balance, or the caller is a different account.

4.9 Experiment 3: Viva Questions with Answers

1. **What is the aim of the Bank contract?**
   To deposit Ether, withdraw Ether and read the caller's recorded balance.
2. **What is a smart contract?**
   A program deployed to a blockchain and executed when called under the network's rules.
3. **What is Solidity?**
   A high-level programming language commonly used to write EVM smart contracts.
4. **What is a mapping?**
   A key-value data structure that associates keys with values.
5. **Why is the mapping key an address?**
   Because the contract records a separate balance for each Ethereum address.
6. **What does `public` do for the mapping?**
   It generates an external getter for reading a value by key.
7. **Why is `deposit()` payable?**
   So the function can receive Ether.
8. **What is `msg.sender`?**
   The address that called the current function.

9. **What is `msg.value`?**
   The amount of Ether attached to the current call, in Wei.
10. **What is Wei?**
    The smallest denomination of Ether; 1 ETH equals 10^{18} Wei.
11. **Why check that the deposit is greater than zero?**
    To reject empty deposits.
12. **Why is `amount` a function argument in `withdraw()`?**
    It specifies the amount the caller wants to withdraw, in Wei.
13. **Why check the recorded balance before withdrawal?**
    To prevent the caller from withdrawing more than their contract-recorded balance.
14. **What does `require` do?**
    It checks a condition and reverts the call if the condition is false.
15. **Why update the balance before sending Ether?**
    It follows checks-effects-interactions and helps reduce reentrancy risk.
16. **What is reentrancy?**
    A situation where an external call re-enters a contract before an earlier operation has fully completed.
17. **What does `call` return here?**
    A success flag and returned data; the example checks the success flag.
18. **Why is `showBalance()` view?**
    It reads state but does not modify it.
19. **Why does `balanceOf(address)` need an address?**
    The getter needs the mapping key whose balance is requested.
20. **What is the difference between ETH and Wei?**
    ETH is the main denomination; Wei is a much smaller unit, with 10^{18} Wei per ETH.
21. **What is gas?**
    A measure of computational work performed by the EVM.
22. **Does the amount transferred alone determine the transaction fee?**
    No. Gas used and effective gas price are the main factors in the fee.
23. **Why should deployment Value be zero in this exercise?**
    The constructor does not need Ether; attaching Value is unnecessary and can cause avoidable problems.
24. **What happens if a transaction reverts?**
    State changes from that execution are rolled back, but an included on-chain failed transaction can still consume gas.

5 Experiment 4: Student Data Using Struct, Array and Fallback

5.1 Aim

To create a Solidity contract that stores student records using a structure and a dynamic array, includes fallback handling, and allows records to be added and read.

5.2 Theory

1. **Structure:** a `struct` groups related fields into a single user-defined record.
2. **Dynamic array:** stores a sequence of elements and can grow as new elements are appended.
3. **Indexing:** Solidity array indices start at zero.
4. **`push`:** appends a new element to a dynamic storage array.
5. **Fallback:** `fallback()` executes when a call's calldata does not match a declared function selector. It must be payable to accept Ether through this route.
6. **Receive:** `receive()` executes for a plain Ether transfer with empty calldata when it is defined. It is included in this sample to make the distinction clear.
7. **Gas observation:** deployment and calls that modify records consume gas. View functions called as local reads generally do not require an on-chain transaction fee.

5.3 Solidity Program: `Student.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Student {
    struct StudentData {
        uint256 rollNo;
        string name;
        uint256 marks;
    }

    StudentData[] public students;

    function addStudent(
        uint256 _rollNo,
        string memory _name,
        uint256 _marks
    ) public {
        students.push(StudentData(_rollNo, _name, _marks));
    }

    function getStudent(uint256 index)
        public
        view
        returns (uint256, string memory, uint256)
    {
        require(index < students.length, "Invalid student index");
        StudentData memory s = students[index];
        return (s.rollNo, s.name, s.marks);
    }

    function getStudentCount() public view returns (uint256) {
        return students.length;
    }
```

```
34
35      fallback() external payable {
36          // Handles calls that do not match a declared function.
37      }
38
39      receive() external payable {
40          // Handles direct Ether transfers with empty calldata.
41      }
42  }
```

5.4 Code Explanation

- `struct StudentData` defines three fields: roll number, name and marks.
- `StudentData[] public students` declares a dynamic array stored in contract state.
- `addStudent` receives the three values and appends a new record using `push`.
- `string memory _name` means the input string is handled in temporary memory during the call.
- `getStudent(index)` first checks that the index is valid, copies the selected record to memory and returns its fields.
- `getStudentCount()` returns the number of stored records.
- The automatically generated getter for the public array can read individual entries by index; `getStudent()` provides a convenient combined return value.
- `fallback()` and `receive()` are special functions and do not have ordinary user-defined names or parameters.

5.5 Step-by-Step Procedure in Remix VM

1. Open https://remix.ethereum.org/.
2. Create a new file named `Student.sol`.
3. Paste the program and save it.
4. Open *Solidity Compiler*, select a compiler compatible with `^0.8.20`, and compile the file.
5. Open *Deploy & Run Transactions*, select *Remix VM*, select the `Student` contract, keep Value at `0 Wei`, and deploy.
6. Expand `addStudent`. Enter `101`, `Amit`, and `85`, then execute the transaction.
7. Call `getStudentCount()`; it should return 1.
8. Call `getStudent(0)`; it should return roll number 101, name Amit and marks 85.
9. Add another record, such as `(102, "Neha", 91)`. The count should become 2.
10. Call `getStudent(1)` to read the second record.
11. Call `getStudent(10)` to test the bounds check; it should revert with `Invalid student index`.

Fallback testing is not necessary to verify student record storage. Avoid sending Ether accidentally. If an instructor specifically asks you to demonstrate fallback, use a controlled test call with unmatched calldata and understand the transaction interface first.

5.6 Gas and Transaction Observation

1. In Remix VM, inspect the transaction log after deployment and after `addStudent()` to view simulated gas information.
2. Remember that Remix VM results are local simulations, not public Ethereum transactions.
3. For a public testnet deployment, connect MetaMask to Sepolia, choose *Injected Provider* in Remix, verify that the selected network is Sepolia, deploy and approve the wallet prompts.
4. After a transaction is mined on Sepolia, copy its transaction hash and open https://sepolia.etherscan.io/.
5. Inspect status, block number, gas used, gas price and fee where the explorer displays them.

5.7 Expected Observations

- After inserting one student, `getStudentCount()` returns 1.
- `getStudent(0)` returns the first student's data.
- After inserting a second student, the count returns 2 and `getStudent(1)` returns the second record.
- An invalid index triggers the `require` check and reverts.
- Deployment and insertion are state-changing operations; read-only calls are normally performed without an on-chain transaction when invoked as view reads.

5.8 Experiment 4: Viva Questions with Answers

1. **What is the aim of this experiment?**
   To store and retrieve student data using a struct and dynamic array, and include fallback handling.
2. **What is a struct in Solidity?**
   A user-defined type that groups multiple related fields.
3. **Why use a struct for student data?**
   It keeps roll number, name and marks together as one logical record.
4. **What is an array?**
   An ordered collection of elements of a specified type.
5. **What is a dynamic array?**
   An array whose length can change during execution.
6. **Why is `students` dynamic?**
   The number of student records is not fixed in advance.
7. **What does `push` do?**
   It appends a new element to a dynamic storage array.
8. **What is the first valid array index?**
   Zero.
9. **What does `students[0]` refer to?**
   The first student record.
10. **Why is `addStudent()` state-changing?**
    It appends a record to the persistent array.
11. **What does `getStudentCount()` return?**
    The current number of student records.

12. **Why is `getStudent()` marked view?**
    It reads and returns data without modifying contract state.
13. **What does `memory` mean for `_name`?**
    The string is held in temporary memory during the call.
14. **What is the difference between memory and storage?**
    Memory is temporary during execution; storage is persistent contract state.
15. **What is fallback()?**
    A special function invoked when calldata does not match a declared function selector.
16. **When can fallback receive Ether?**
    When it is payable and the call reaches fallback rather than a defined receive function for an empty-calldata transfer.
17. **What is receive()?**
    A special payable function used for plain Ether transfers with empty calldata.
18. **What is the difference between fallback() and receive()?**
    receive() handles empty-calldata Ether transfers; fallback() handles unmatched calls and may accept Ether if payable.
19. **Why does the sample have both functions?**
    To demonstrate both special-function cases separately.
20. **What happens when an invalid index is passed?**
    The `require` condition fails and the call reverts.
21. **Why is the array public?**
    Solidity generates a getter that can read an element by index.
22. **Does adding a student use gas?**
    Yes. It changes persistent state and is a state-changing operation.
23. **Does reading a student always cost a transaction fee?**
    A view read called locally normally does not create an on-chain transaction; a transaction-based invocation is different.
24. **What is the purpose of the SPDX line?**
    It declares the source-code license identifier for tooling and verification.
25. **Why must the compiler version be compatible?**
    Solidity syntax and features depend on the compiler version range declared by the source.

6 Quick Revision Tables

6.1 Important Solidity Terms

Term	Meaning
`contract`	Defines a smart contract.
`mapping`	Associates keys with values.
`struct`	Groups related fields into one record.
`array`	Stores an ordered sequence of elements.
`payable`	Allows a function to receive Ether.
`msg.sender`	Address calling the function.
`msg.value`	Ether amount attached to the current call, in Wei.
`require`	Checks a condition and reverts if false.

`view`	Reads state without modifying it.
`fallback()`	Handles unmatched function calls.
`receive()`	Handles plain Ether transfers with empty calldata.
`gas`	Measure of computational work.

6.2 Useful Conversions

$$1 \text{ ETH} = 10^{18} \text{ Wei} \qquad 1 \text{ ETH} = 10^{9} \text{ Gwei}$$

$$\text{Fee in ETH} = \frac{\text{gas used} \times \text{gas price in Gwei}}{10^{9}}$$

Example: if a transaction uses 21,000 gas at 20 Gwei, the approximate fee is

$$\frac{21{,}000 \times 20}{10^{9}} = 0.00042 \text{ ETH}.$$

7 Final Practical Checklist

- I can explain each line of both contracts.
- I can compile and deploy each contract in Remix VM.
- I can deposit and withdraw using the correct Value field and Wei argument.
- I can add at least two student records and retrieve them using indices 0 and 1.
- I know why invalid withdrawals and invalid student indices revert.
- I can distinguish Remix VM from Sepolia.
- I can explain gas used, gas price and transaction fee.
- I will never disclose a Secret Recovery Phrase or private key.